

1. Introduction

The goal of this assignment was to create a memory-efficient program that indexes large text files by building a word frequency map without loading the entire file into memory. The program must:

Process files using a fixed-size buffer (256 KB - 1024 KB)

Create separate indexes for different versions of files

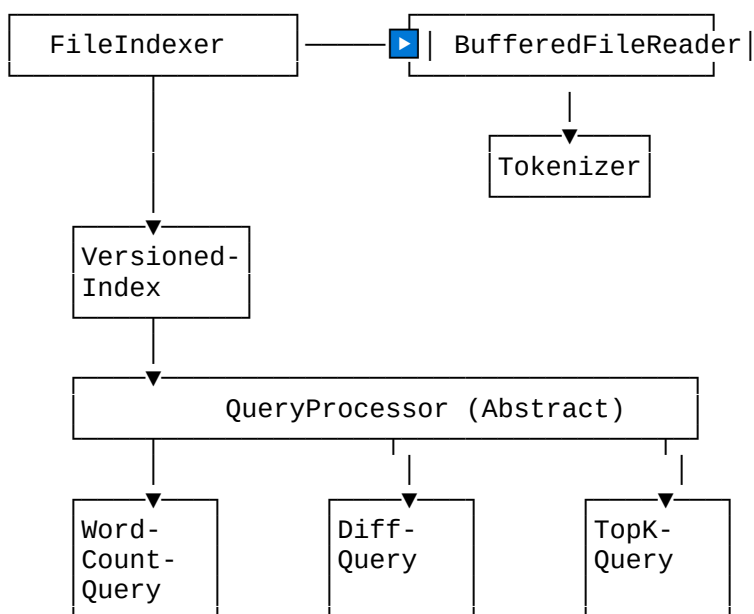
Support three query types: word count, top-K frequent words, and difference between versions

Use object-oriented design with inheritance, polymorphism, templates, and exception handling

2. Design and Approach

2.1 Overall Architecture

The program follows a modular object-oriented design with clear separation of concerns:



2.2 Key Design Decisions

Fixed-size buffer: Prevents loading entire file into memory

Case-insensitive matching: All words converted to lowercase

Separate indexes per version: Each version maintains its own frequency map

Polymorphic queries: All query types derive from abstract base class

Exception handling: Graceful error recovery for file errors, invalid inputs

3. Class Descriptions

3.1 QueryProcessor (Abstract Base Class)

Purpose: Defines interface for all query types

Key members: version (string), execute(), displayResult()

Why abstract: Enforces that all derived queries implement execution and display

3.2 Derived Query Classes

Class	Purpose	Key Functionality
WordCountQuery	Find frequency of a word	Searches index for specific word
DiffQuery	Compare word across versions	Computes difference between two indexes
TopKQuery	Find most frequent words	Sorts and returns top K words

3.3 BufferedFileReader

Purpose: Reads file in chunks using fixed-size buffer

Key members: buffer, bufferSize, bufferPosition, bytesInBuffer

Key methods: fillBuffer(), getNextChar(), isEOF()

Why: Core to memory efficiency - never loads entire file

3.4 Tokenizer

Purpose: Extracts words from buffer content

Key features:

Identifies alphanumeric sequences as words

Converts to lowercase for case-insensitive matching

Handles tokens split across buffer boundaries using leftover string

Key method: tokenize()

3.5 VersionedIndex

Purpose: Stores word frequencies for a specific version

Data structure: unordered_map<string, size_t> for O(1) lookups

Key methods: addWord(), getWordFrequency(), getTopK()

3.6 FileIndexer

Purpose: Coordinates the indexing process

Key members: Map of version names to indexes

Key method: indexFile() - processes file and builds index

3.7 ArgumentParser

Purpose: Parses and validates command-line arguments

Key methods: getString(), getInt(), hasKey()

3.8 Helper Classes

IndexEntry<T> (Template class): Demonstrates template usage

StringUtils: Demonstrates function overloading with toLower()

4. File Processing with Fixed-Size Buffer

4.1 How it Works

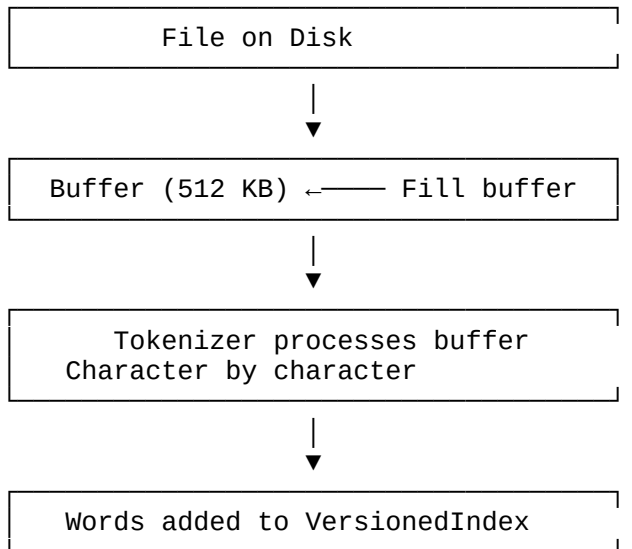
Buffer Initialization:

Buffer size set between 256-1024 KB (user specified)

Memory allocated once, reused throughout

Reading Process:

text



Buffer Refilling:

When buffer position reaches end, fillBuffer() reads next chunk

Continues until EOF

4.2 Handling Split Tokens

When a word is cut across buffer boundaries:

text

Buffer 1: "error debug er"

↑ word split here

Buffer 2: "ror info warning"

Solution:

Tokenizer maintains a leftover string

Incomplete word at end of buffer saved to leftover

Prepended to next buffer's tokens

4.3 Memory Efficiency

Total file size: 1 GB (example)

Memory used: Only ~512 KB (buffer) + space for unique words

Independent of file size: Program can handle files of any size

5. Query Processing

5.1 Word Count Query

Searches the version's frequency map

Returns count (0 if word not found)

Time complexity: $O(1)$

5.2 Top-K Query

Converts map to vector of pairs

Sorts by frequency (descending) then alphabetically

Returns first K elements

Time complexity: $O(n \log n)$ where n = unique words

5.3 Difference Query

Gets frequency from both versions

Computes absolute difference

Time complexity: $O(1)$

6. C++ Features Demonstrated

Requirement Implementation

4+ user-defined classes 8 classes implemented

Inheritance QueryProcessor as abstract base, 3 derived classes

Runtime polymorphism Virtual functions execute() and displayResult()

Function overloading StringUtils::toLowerCase() for string and char*

Exception handling try-catch blocks for file errors, invalid args

Templates IndexEntry<T> class template

STL containers unordered_map, vector, string

7. Assumptions and Observations

7.1 Assumptions Made

Word definition: Only alphanumeric characters (a-z, A-Z, 0-9)

Case handling: All comparisons case-insensitive

Buffer size: Must be between 256-1024 KB as specified

File format: Text files (binary files not supported)

Version names: User-provided, must be unique per execution

7.2 Observations

Performance:

Indexing time proportional to file size

Memory usage grows only with unique words count

Top-K queries slower with many unique words due to sorting

Limitations:

Very large number of unique words (>1 million) may cause memory issues

No persistence between program runs (indexes lost after exit)

Edge Cases Handled:

Empty files

Files with no alphanumeric characters

Words split across buffer boundaries

Very long words (> buffer size)

8. Sample Run

Input:

text

```
.\analyzer.exe --file sample.txt --version v1 --buffer 512 --query word --word error
```

Output:

text

Indexed file: sample.txt

Buffer size: 512 KB

Indexing time: 0.023 seconds

=== Query Results ===

Version: v1

Word: 'error' appears 42 times

Allocated buffer size: 512 KB

Total execution time: 0.025 seconds

9. Conclusion

The program successfully implements a memory-efficient file indexer that:

Processes files of any size using fixed buffer

Maintains separate indexes for multiple versions

Answers three types of queries correctly

Demonstrates strong object-oriented design in C++

Handles errors gracefully

All assignment requirements regarding memory constraints, buffer size, version management, and C++ features have been met.